

Livello del linguaggio macchina

FORMATI DI ISTRUZIONE

Formato di istruzione

- Specifica il significato di ciascun bit che compone una istruzione
- Istruzione suddivisa in campi
 - Campo OPCODE
 - Zero o più campi ADDRESS
 - Ulteriori campi in architetture più complesse (prefissi, modo, etc.)
- Lunghezza di una istruzione:
 - Minore è il numero di bit di una istruzione, meno memoria essa occupa
 - Minore è il numero di bit per istruzione, meno istruzioni possono essere rappresentate
 - Lunghezza fissa o variabile

Formati di istruzione

- La variabilità dei formati d'istruzione dipende
 - dalla natura delle operazioni (che possono essere a zero, uno, due o più operandi)
 - ma anche da scelte architetturali
- Es. una operazione di somma potrebbe essere realizzata attraverso una istruzione:
 - a **3 indirizzi** `ADD A, B, C` $C \leftarrow A+B$
 - a **2 indirizzi** `ADD A, B` $A \leftarrow A+B$
 - a **1 indirizzo** `ADD A` $AC \leftarrow AC+A$
- Esistono architetture basate sullo **stack**, con istruzioni log/arit **prive di operandi**
 - Es. `IADD` equivale a:
`POP R1; POP R2; ADD R1, R2; PUSH R2`



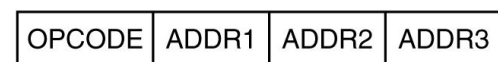
(a)



(b)



(c)



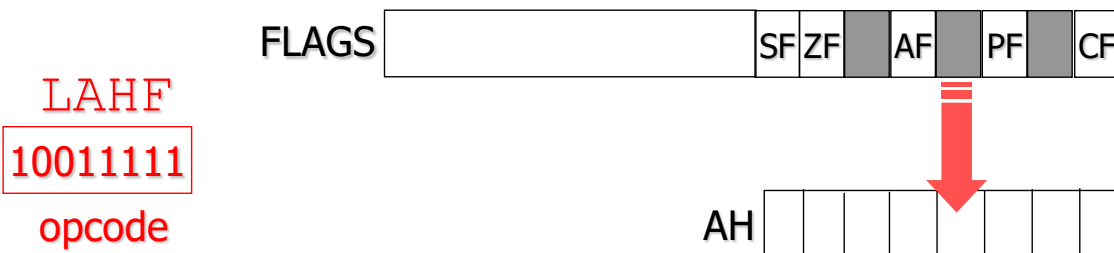
(d)

Formati d'istruzione dell'8086

- Nell'8086 si usa un **formato d'istruzione a lunghezza variabile**
 - le istruzioni usano un numero variabile di byte, in dipendenza dal numero di operandi espliciti che utilizzano
 - Ogni istruzione può avere **zero, uno o due** riferimenti a operandi, e ciascun riferimento può occupare multipli o sottomultipli di un byte

Formati d'istruzione dell'8086

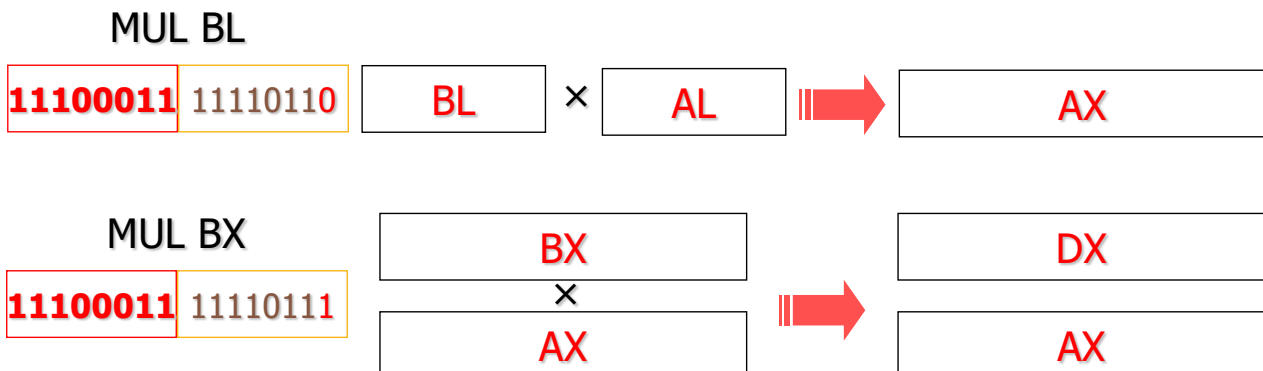
- Esempio di istruzione a **zero operandi**:
 - LAHF**: copia la parte bassa del registro FLAGS (sorgente) in AH (destinazione)
 - I due operandi sono **impliciti** nel codice operativo



15

Formati d'istruzione dell'8086

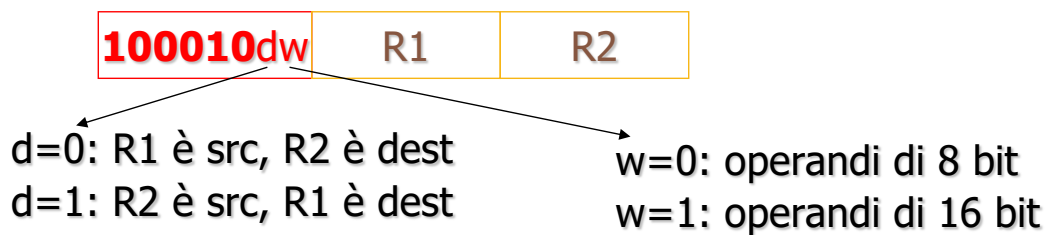
- Esempio di istruzione a **un operando**:
 - MUL op1**: moltiplica il dato specificato dall'operando per un valore contenuto in (tutto o parte di) AX
 - Uno dei due operandi (l'accumulatore) è implicito nel codice operativo



17

Formati d'istruzione dell'8086

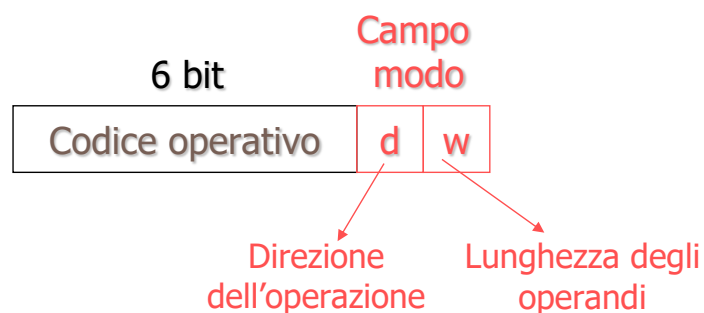
- Esempio di istruzione a **due operandi**:
 - MOV *dest*, *src***: copia il contenuto di *src* (sorgente) in *dest* (destinazione)
 - dest* e *src* possono essere un registro generale o una locazione di memoria, ma non entrambi locazioni di memoria



18

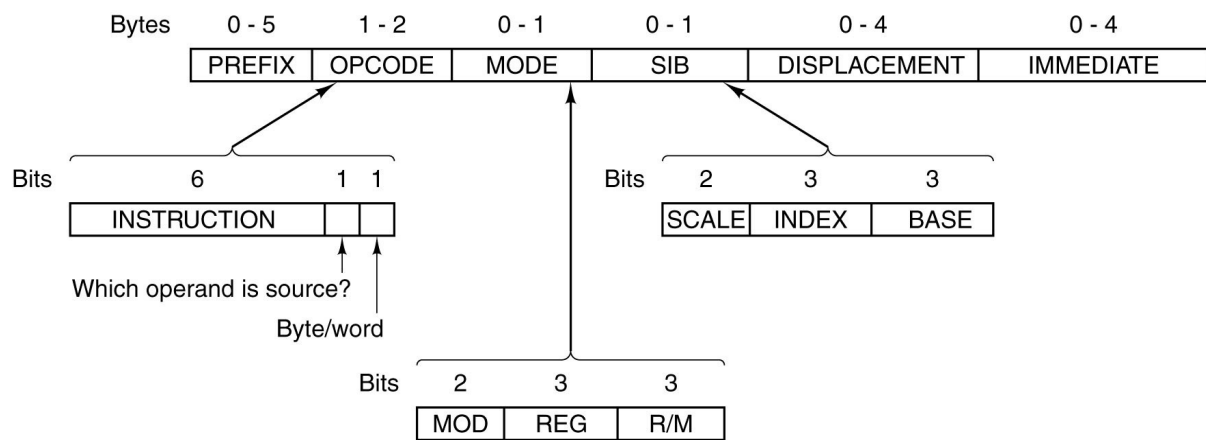
Formato di istruzione ortogonale

- Il formato di istruzione è **ortogonale** se l'istruzione contiene un campo "modo" che specifica alcune informazioni (es. la lunghezza degli operandi, il numero di operandi, il tipo di indirizzamento, ...)
 - Le **istruzioni dell'8086** hanno un formato ortogonale



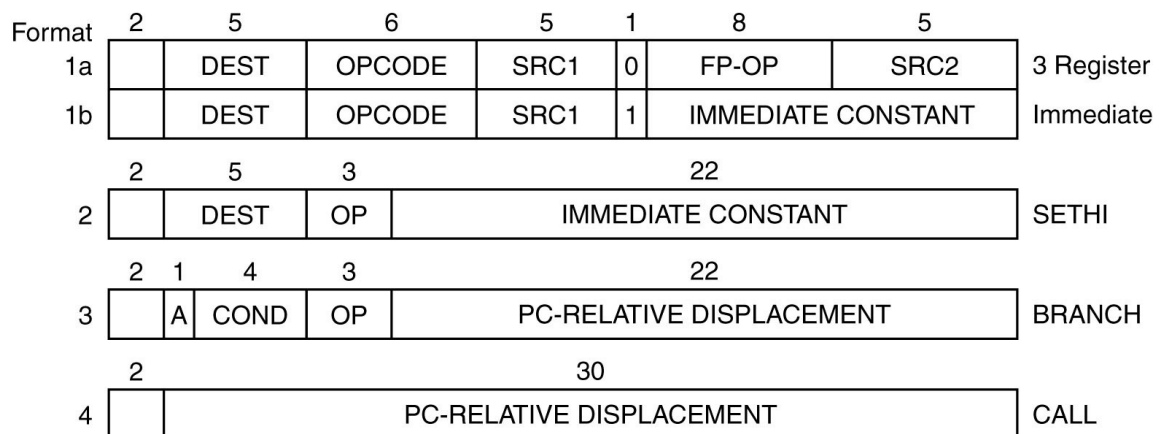
19

Formati delle istruzioni di Pentium 4



29

Formati delle istruzioni di UltraSPARC III



30

Livello del linguaggio macchina

MODALITÀ DI INDIRIZZAMENTO

Modalità di indirizzamento

- Specifica l'interpretazione dei bit dei campi indirizzo
- Diverse modalità di indirizzamento
- Non tutte le modalità sono implementate in tutte le architetture
 - Architetture più semplici (es. RISC) hanno modalità di indirizzamento limitate
- Maggiori modalità di indirizzamento →
 - Maggiore flessibilità
 - Decodifica più complessa
 - Istruzioni più lunghe
- Istruzione di riferimento: **MOV *op1*, *op2*** (copia il contenuto di *op1* in *op2*)
 - Le modalità di indirizzamento generalmente si riferiranno all'interpretazione di *op2*

Indirizzamento immediato

- Il campo indirizzo specifica l'operando stesso
 - Non è un indirizzamento vero e proprio
- Espressione (tramite mov): **MOV Reg, x**
- L'esecuzione dell'istruzione è immediata
 - Non richiede operand fetch
- Uso molto limitato (utilizzo di piccoli valori costanti)
- Esempi (**8086**):
 - **MOV AL, 00** carica in AL il valore 0000 0000
 - **MOV CX, 04** carica in CX il valore 0000 0000 0000 0100
 - **MOV AH, 0FFh** (valore esadecimale "piccolo")
 - **MOV BX, 1F3Ah** (valore esadecimale "grande")
 - **MOV DX, 11d** (valore decimale)
 - **MOV AX, 11b** (valore binario)

35

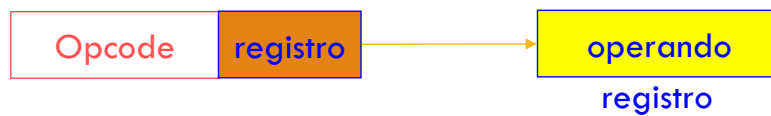
Indirizzamento implicito

- Il campo indirizzo non è specificato, ma è assunto sulla base dell'istruzione
- Espressione (tramite push): **PUSH R1**
 - Memorizza nello stack il contenuto di R1
 - Indirizzamento implicito allo **stack pointer**
 - Equivale alle istruzioni
 - **DEC SP**
 - **MOV [SP], R1**
- Espressione (tramite mul): **MUL BX**
 - Effettua la moltiplicazione (**BX × AX**)
 - Indirizzamento implicito al **registro AX** (accumulatore)

36

Indirizzamento a registro

- Il campo indirizzo specifica un registro della CPU
- Espressione (tramite mov): **MOV Reg1, Reg2**
 - Memorizza in Reg1 il contenuto del registro Reg2
 - Nessun accesso in memoria → velocità di esecuzione
 - Istruzioni compatte
- Utilizzato per la gestione di variabili memorizzate in registri



39

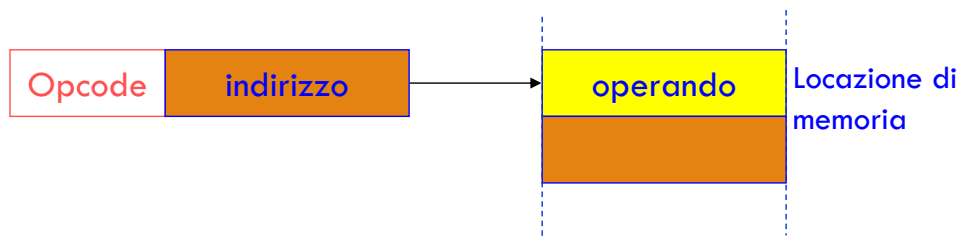
Indirizzamento a registro (8086)

- Esempi:
 - **MOV DX, AX** → copia in DX il contenuto di AX (operandi da 16 bit)
 - **MOV AH, BL** → copia in AH il contenuto di BL (operandi da 8 bit)
 - **MOV DL, AX** → NON CONSENTITA!
 - **MOV DX, AL** → NON CONSENTITA!

40

Indirizzamento diretto

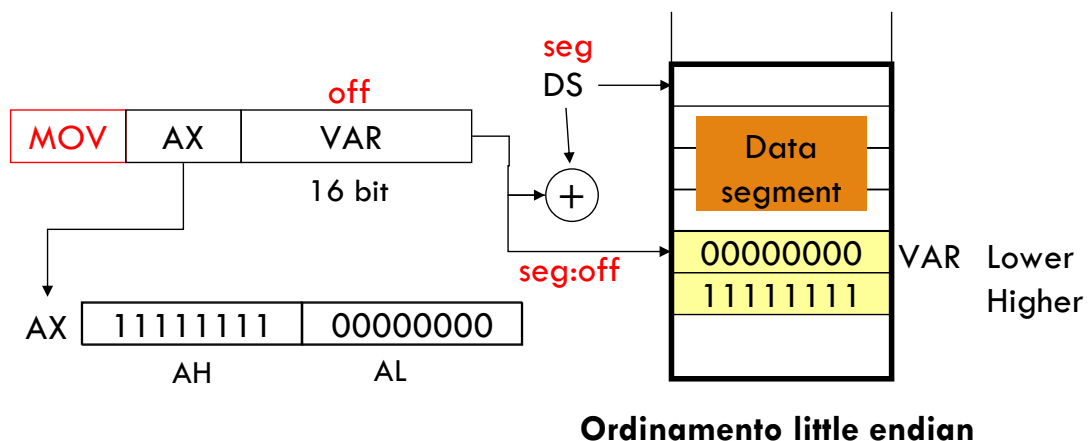
- Il campo indirizzo specifica l'indirizzo in memoria in cui è allocato il dato richiesto
- Espressione (tramite mov): **MOV Reg1, [addr]**
 - Memorizza in Reg1 il contenuto in memoria a partire dall'indirizzo addr
 - Il numero di parole trasferite dalla memoria alla CPU dipende dalla profondità di Reg1
 - In Assembly gli indirizzi si specificano simbolicamente attraverso **variabili** (es. MOV Reg1, var)
- Indirizzamento adatto a gestire variabili scalari



41

Indirizzamento diretto (8086)

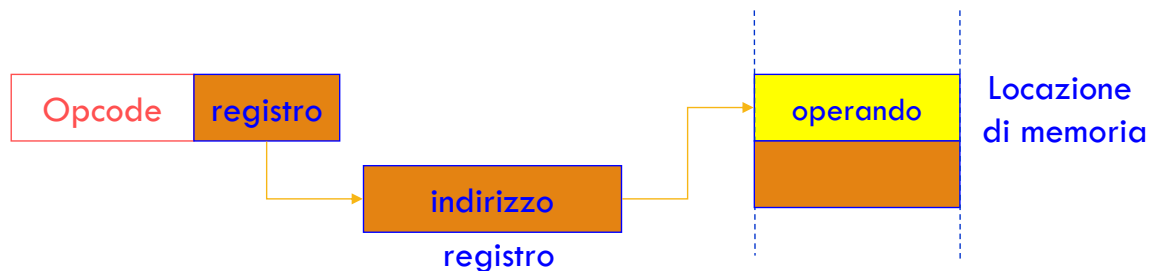
- Il campo indirizzi dell'istruzione contiene l'**offset** all'interno del segmento
 - Il riferimento al segmento dati è implicito
- Esempio: **MOV AX, VAR**



42

Indirizzamento a registro indiretto

- Il campo indirizzo specifica un registro che contiene l'indirizzo del dato richiesto
- Espressione (tramite mov): **MOV Reg1, [Reg2]**
 - Memorizza nel registro Reg1 il contenuto di memoria a partire dall'indirizzo specificato nel registro Reg2
 - Istruzioni compatte
- Utilizzato per la gestione dei “puntatori”, ossia variabili che contengono indirizzi



43

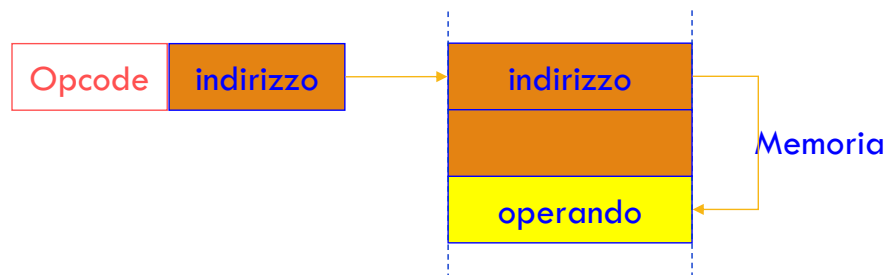
Indirizzamento a registro indiretto (8086)

- Il registro riferito nel campo indirizzi dell'istruzione contiene l'**offset** dell'operando all'interno del segmento dei dati
- Si utilizza il registro **base** (BX) o un registro **indice** (DI, SI)
 - Indirizzamento **indiretto con registro base**
 - Indirizzamento **indiretto con registro indice**
- Esempio:
 - **MOV AL, [BX]** → copia in AL il valore contenuto nell'indirizzo di memoria puntato da BX

44

Indirizzamento indiretto

- Il campo indirizzo specifica un indirizzo a una locazione di memoria che contiene l'indirizzo del dato richiesto
- Espressione (tramite mov): **MOV R1,[[addr]]**
 - Memorizza in R1 il contenuto di memoria a partire da un altro indirizzo specificato in memoria
 - Doppio accesso in memoria → implementazione difficoltosa → scarso utilizzo
 - Non previsto nell'8086
 - Si può ottenere con un indirizzamento diretto + uno a registro indiretto



45

Indirizzamento indicizzato

- Il campo indirizzo specifica un registro (**indice**) + una costante di base
- Espressione (tramite mov): **MOV R1, addr[R2]**
MOV R1, [addr+R2]
 - Memorizza in R1 il contenuto di memoria a partire dall'indirizzo che si ottiene sommando alla base addr il contenuto del registro R2
 - In Assembly si usa la forma simbolica MOV R1, VAR[R2]
 - La somma viene effettuata in fase di decodifica
- Utilizzato per la gestione degli array, dove la base indica l'indirizzo di inizio dell'array, mentre il registro contiene l'indice dell'array (moltiplicato per la dimensione dell'elemento)

47

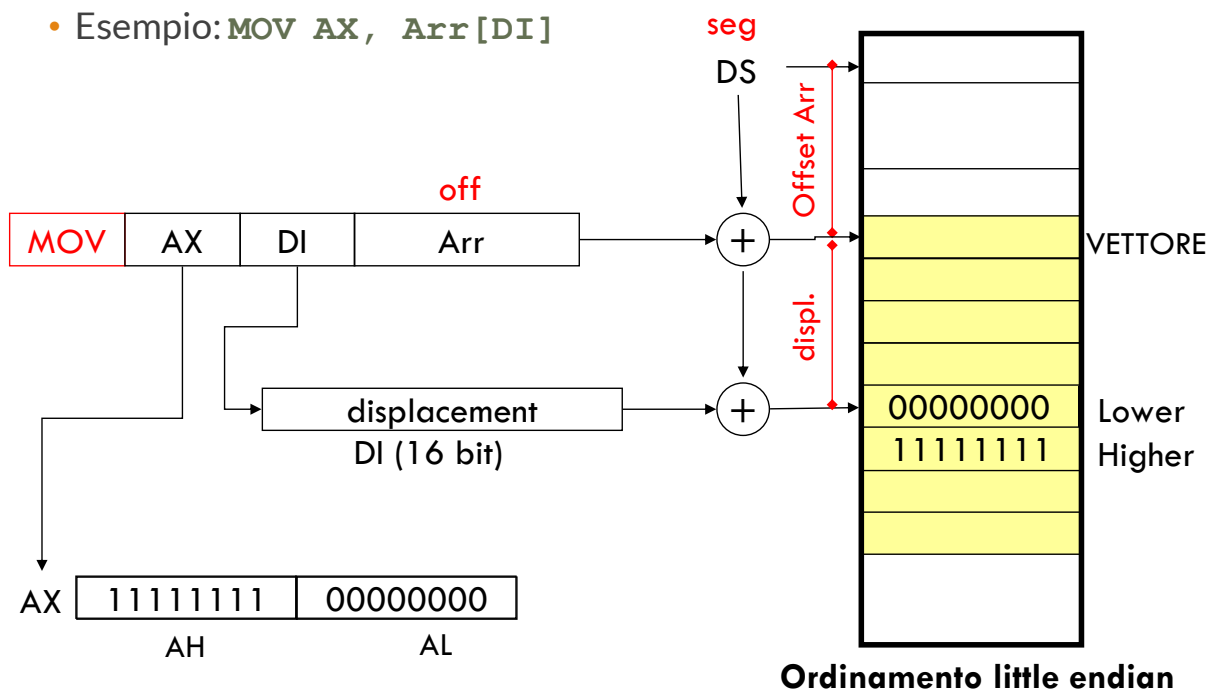
Indirizzamento indicizzato (8086)

- L'offset dell'operando si ottiene sommando il contenuto di un registro indice (SI o DI) a un valore contenuto nell'istruzione
- Adatto per accedere agli elementi di un vettore
 - La costante di base corrisponde all'offset del 1° elemento del vettore
 - Il registro indice contiene il **displacement** rispetto al primo elemento, cioè l'indice dell'elemento del vettore
 - `MOV AH, var_array[DI]`
 - `MOV AH, var_array[SI]`

48

Indirizzamento indicizzato (8086)

- Esempio: `MOV AX, Arr[DI]`



49

Indirizzamento a registro base

- Il campo indirizzo specifica un registro (base) + una costante di spiazzamento
- Esempio: `MOV R1, [R2+x]`
`MOV R1, R2[x]`
 - Memorizza in R1 il contenuto di memoria a partire dall'indirizzo che si ottiene sommando alla base contenuta in R2 lo spiazzamento x
 - La somma viene effettuata in fase di decodifica
- Utilizzato per la gestione dei record (struct), dove il registro base indica l'indirizzo di inizio del record, mentre lo spiazzamento indica la posizione relativa di un campo (membro)
- Nell'8086: BX registro base di accesso alla memoria
 - `MOV AL, [BX] + 4`

50

Indirizzamento a registro base + indice (o indirizzamento esteso)

- Il campo indirizzo specifica un **registro (base)** + **un registro indice** + un'eventuale **costante di spiazzamento**
- Esempio: `MOV R1, [R2+R3+x]`
 - Memorizza in R1 il contenuto di memoria a partire dall'indirizzo che si ottiene sommando alla base contenuta in R2 l'indice contenuto in R3 più lo spiazzamento x
 - La somma viene effettuata in fase di decodifica
- Modalità di indirizzamento molto flessibile
 - Es. nell'8086: `MOV AL, [BX+SI+4]`
- Utilizzata per la gestione di array di record (file) oppure di record con campi array
- Difficile da utilizzare
 - Utilizzata soprattutto dai compilatori

53

Indirizzamento relativo

- Il campo indirizzo specifica un offset rispetto al contenuto di un registro (implicito)
 - Tipicamente il PC
- Esempio: **JE -5**
 - Se è verificata la condizione di uguaglianza, salta all'indirizzo che si ottiene sottraendo 5 dal Program Counter
 - Equivale a **ADD PC, -5** ma **PC non è accessibile** a livello ISA!
 - Istruzioni compatte (utili per salti a breve distanza)
- Usato per i salti

Livello del linguaggio macchina

PARCO ISTRUZIONI

Tipi di istruzione

- Esistono molte istruzioni in una ISA, e ISA diverse hanno istruzioni molto diverse fra loro
 - Tuttavia è possibile stabilire categorie (tipi) di istruzioni, e tali tipi sono pressoché identici per tutte le ISA
- Tipi di istruzione
 - Istruzioni di trasferimento
 - Istruzioni logico/aritmetiche
 - Istruzioni di confronto e salto
 - Istruzioni per invocare procedure
 - Istruzioni di ciclo
 - Istruzioni per l'Input/Output

64

Istruzioni di trasferimento dati

- Copiano dati da una locazione a un'altra, solitamente:
 - Da registro a registro
 - Da costante a registro
 - Da registro a memoria
 - Da memoria a registro
 - Mai da memoria a memoria (per semplicità)
 - Mai da registro a costante o da costante a costante (perché non ha senso)
- Per ogni istruzione di trasferimento, si deve specificare la sorgente (costante, registro o indirizzo) e la destinazione (registro o indirizzo)

65

Istruzioni Logico/Aritmetiche

- Utilizzano funzioni specifiche della ALU
 - Sempre SOMMA, SOTTRAZIONE e COMPLEMENTO (a 2)
 - Quasi sempre MOLTIPLICAZIONE e DIVISIONE (con o senza segno)
 - Non sono presenti nel **computer Hack**
 - Quasi sempre operazioni complesse (trigonometriche, polinomiali, etc.) in floating point
 - Non sono presenti **nell'8086**
 - Sempre operazioni logiche di base AND, OR, NOT
 - Quasi sempre XOR
 - Raramente NOR, NAND, XNOR
 - Spesso operazioni di SHIFT e ROTAZIONE

67

Uso delle istruzioni logiche

AND

- Serve a estrarre (spacchettare) sottostringhe di bit
- Es: **AND R1, 00F0h**
 - estrae da R1 il secondo nibble meno significativo (ponendo tutti gli altri bit a zero)

0 1 1 0 1 0 1 1 1 0 0 1 0 1 1 1

AND

0 0 0 0 0 0 0 0 0 1 1 1 0 0 0 0



0 0 0 0 0 0 0 0 0 1 0 0 1 0 0 0

OR

- Serve a unire (**impacchettare**) sottostringhe di bit
- Es: **OR R1, 000Bh** (nibble meno significativo di R1 pari a 0)
 - memorizza B (hex) nel nibble meno significativo in R1

0 1 1 0 1 0 1 1 1 0 0 1 0 0 0 0

OR

0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1



0 1 1 0 1 0 1 1 1 0 0 1 1 0 1 1

70

Istruzioni di confronto e di salto

- Tutte le istruzioni logico/aritmetiche modificano la PSW
 - Es SUB R1, R1 imposta il bit Z (zero) della PSW a 1 (il risultato è nullo)
- I bit della PSW si possono utilizzare per modificare il flusso di esecuzione di un programma attraverso **salti condizionati**
 - Realizzazione dei costrutti IF THEN ELSE
- Le istruzioni di confronto consentono di modificare la PSW senza alterare il contenuto dei registri:
 - CMP = SUB senza memorizzare il risultato (8086)
 - TEST = AND senza memorizzare il risultato (8086)
- **Salto incondizionato**: salta indipendentemente dalla PSW

72

Istruzioni di ciclo

- I cicli (**iterazioni**) possono essere implementati mediante istruzioni di salto, di solito associate a istruzioni di incremento o decremento
- Alcune ISA mettono a disposizione istruzioni dedicate, per risparmiare memoria e per essere più veloci
- Es. (8086):
 - **LOOP indirizzo** = decrementa CX e, se CX è diverso da zero, salta a *indirizzo*
 - realizzazione efficiente di cicli FOR semplici
- Per i cicli WHILE e DO...WHILE (REPEAT...UNTIL) si usano i salti condizionati

74

Istruzioni di invocazione procedura

- L'invocazione della procedura equivale a un salto incondizionato, ma la procedura chiamata non conosce a priori dove riprendere l'esecuzione una volta terminata
- Istruzione dedicata per l'invocazione **CALL**, che memorizza il valore del PC prima di modificarlo
 - Nel PC si scrive l'indirizzo relativo alla prima istruzione della procedura
- Istruzione **RET**, al termine della procedura, recupera il valore del PC per riprendere l'esecuzione del chiamante da dove è stato interrotto

Interrupt

- Un interrupt è un'azione che interrompe il normale flusso di esecuzione di un programma
- Interrupt asincroni: non sono previsti dal programmatore
 - Eventi generati da dispositivi esterni
 - Eventi eccezionali (calo di tensione, reset, ...)
 - Eccezioni software (divisione per zero, accesso a memoria proibita, esecuzione di istruzioni privilegiate in modo utente)
- Interrupt sincroni: previsti dal programmatore mediante istruzioni specifiche (**INT**)
 - Interfacciamento con il sistema operativo o il BIOS